W08 — Monaco Editor for Custom Prompt Templates

Epic: E-008 BYO-LLM / Prompt Customization **Screen**: JudgeConfig, "Compiled Prompt" card (Screen 2 in E-008 UX spec) **Depends on**: W1 (canonical prompt-template var schema in `backend/src/services/prompt-template.ts`) **Coordinates with**: W1 (prompt renderer contract) **Status**: Implementation-ready

---

1. Goal

Replace the read-only compiled-prompt `<pre>` block in `JudgeConfig.tsx` (lines 202–213) with an editable Monaco-based authoring surface for the System and User prompt templates. Saved templates are versioned into the existing `project_judge_config_history`, validated both client- and server-side against the canonical W1 variable set, previewable against a sample claim, and resettable to the built-in defaults.

Canonical variables (from W1, must stay in lockstep):

Var	Type	Required in	Notes
`{{claim}}`	string	user	Truncated to 4,000 chars upstream.
`{{evidence}}`	array<{title,snippet,url?}>	user	Rendered as `<<<EVIDENCE_N>>>` blocks.
`{{guideline}}`	string (optional)	system	Truncated to 3,000 chars.
`{{tier}}`	string	user	Routing tier label.
`{{claim_id}}`	string (UUID)	user	For traceability in logs.

Canonical output schema (6 rubric fields — must remain in system template): `verdict`, `confidence`, `rubric.factuality`, `rubric.relevance`, `rubric.citation`, `rubric.safety`, `reason`.

---

2. Bundle & dependency strategy

2.1 Add `@monaco-editor/react`

- Package: `@monaco-editor/react@^4.7.0` (peer: `monaco-editor@^0.52.0`).
- Raw size: monaco-editor unminified ~10 MB; compressed ~2.4 MB gzipped when Vite code-splits it. Loading synchronously in main bundle would roughly triple current JS payload — **unacceptable**.
- **Mitigation**: lazy-load via `React.lazy` + dynamic `import()`; only pulled in when the PM opens the "Edit templates" tab. Vite emits the chunk as a separate async asset.
- **Monaco worker setup**: configure `loader.config({ paths: {...} })` from `@monaco-editor/react`'s `loader` helper OR rely on the default CDN (`https://cdn.jsdelivr.net/npm/monaco-`

editor@0.52.0/min/vs). Use CDN for v1.0 — avoids a Vite worker-asset pipeline change. Document offline caveat in the loading skeleton.

- **CSP**: `vite.config.ts` already permits inline styles; add `cdn.jsdelivr.net` to `script-src` / `worker-src` if a strict CSP is introduced later (not required today — no CSP header yet).
- **Fallback**: if Monaco fails to load (network / CSP), render a plain `<textarea>` with the same `onChange` contract. The validation pipe is identical.

2.2 `package.json` diff

```
   "dependencies": {
     "@supabase/supabase-js": "^2.103.0",
     "@tailwindcss/vite": "^4.2.3",
+    "@monaco-editor/react": "^4.7.0",
     "framer-motion": "^12.38.0",
```

No dev-dep change required. Run `npm install` after merge.

---

3. DB migration

3.1 File: `supabase/migrations/20260422_0001_judge_config_prompt_template.sql`

```
-- Migration: judge_config prompt_template columns
-- Adds optional custom system/user prompt templates to project_judge_config
-- and mirrors them in the _history table so a pinned historical llm_runs
-- row can still resolve its exact template source.

ALTER TABLE public.project_judge_config
  ADD COLUMN IF NOT EXISTS system_prompt_template TEXT,
  ADD COLUMN IF NOT EXISTS user_prompt_template   TEXT;

COMMENT ON COLUMN public.project_judge_config.system_prompt_template IS
  'Optional custom system prompt template. NULL = use built-in prompt '
  'from services/prompt-template.ts. Must contain {{guideline}} when set.';

COMMENT ON COLUMN public.project_judge_config.user_prompt_template IS
  'Optional custom user prompt template. NULL = use built-in. When set, '
  'must contain {{claim}}, {{evidence}}, {{tier}}, {{claim_id}}.';

ALTER TABLE public.project_judge_config_history
  ADD COLUMN IF NOT EXISTS system_prompt_template TEXT,
  ADD COLUMN IF NOT EXISTS user_prompt_template   TEXT;
```

Forward-only (see house rules). No data backfill — existing rows keep `NULL`, which the renderer treats as "use built-in".

3.2 Embedded-postgres parity

Add an equivalent file under `backend/migrations/` if that lane is active; mirror the columns 1:1.

---

4. Shared validation module

4.1 New file: `src/lib/promptTemplateValidation.ts`

Pure, no React/DOM deps so backend can import it via a relative path or duplicate the logic in a sibling file under `backend/src/services/`.

```
export interface ValidationResult {
  valid: boolean;
  missingRequired: string[];   // required vars not present
  unknownVars: string[];       // {{x}} used but not in canonical set
  outputSchemaOk: boolean;     // system prompt references all 6 rubric keys
  errors: string[];            // human-readable messages
}

export const REQUIRED_SYSTEM_VARS = ['guideline'] as const;
export const REQUIRED_USER_VARS   = ['claim', 'evidence', 'tier', 'claim_id'] as const;
export const ALL_VARS = [
  ...REQUIRED_SYSTEM_VARS, ...REQUIRED_USER_VARS,
] as const;

export const REQUIRED_OUTPUT_KEYS = [
  'verdict', 'confidence',
  'factuality', 'relevance', 'citation', 'safety',
  'reason',
] as const;

export function extractVars(template: string): string[] { /* /\{\{\s*(\w+)\s*\}\}/g */ }

export function validateSystemTemplate(tmpl: string): ValidationResult;
export function validateUserTemplate(tmpl: string): ValidationResult;
```

Semantics: - **Missing required** → hard error (block save). - **Unknown var** → warning (allow save but surface in UI). - **Output schema check** (system only): all 7 REQUIRED_OUTPUT_KEYS must appear as literal substrings in the system template.

4.2 Vitest: `src/lib/__tests__/promptTemplateValidation.test.ts`

Cases (≥ 10): 1. Empty string → both required lists missing, not valid. 2. Built-in default system template → valid, no warnings. 3. Built-in default user template → valid. 4. User template missing `{{claim}}` → `missingRequired: ['claim']`, invalid. 5. User template with `{{frobnicate}}` → warns `unknownVars: ['frobnicate']`, still valid if required vars present. 6. System template missing `factuality` output key →

`outputSchemaOk: false`, invalid. 7. `{{ claim }}` (with whitespace) accepted. 8. Case-sensitive: `{{Claim}}` treated as unknown. 9. Duplicate vars (`{{claim}} ... {{claim}}`) → required satisfied, no dupe warning. 10. `extractVars` returns unique values.

Runner: reuse existing vitest config in `backend/` (add a frontend vitest config if not yet present — otherwise run via `backend/test/` since the module is pure). Preferred location: put the test next to the source in `src/lib/__tests__/` and add `vitest` + `@vitest/ui` as devDeps if missing. If frontend vitest is a blocker, **copy** the module to `backend/src/services/promptTemplateValidation.ts` and test there — keep the two files byte-identical; a follow-up refactor extracts a shared package.

---

5. Frontend diff: `src/components/JudgeConfig/JudgeConfig.tsx`

5.1 New types & state (near line 22)

```
 interface CompiledPrompt { text: string; summary?: string; }
+interface PromptTemplates {
+  systemTemplate: string | null;  // null = use default
+  userTemplate:   string | null;
+}
```

Add to the `JudgeConfigData` interface:

```
   fallbackChain: FallbackEntry[];
+  systemPromptTemplate: string | null;
+  userPromptTemplate:   string | null;
 }
```

Add to `DEFAULTS`:

```
-const DEFAULTS: JudgeConfigData = { providerId: '', model: '', ... , fallbackChain: [] };
+const DEFAULTS: JudgeConfigData = { providerId: '', model: '', ... , fallbackChain: [],
+  systemPromptTemplate: null, userPromptTemplate: null };
```

New local state inside component:

```
const [tab, setTab] = useState<'system' | 'user'>('system');
const [showPreview, setShowPreview] = useState(false);
const [editing, setEditing] = useState(false);
```

5.2 Lazy Monaco import (top of file)

```
const MonacoEditor = React.lazy(() =>
  import('@monaco-editor/react').then((m) => ({ default: m.Editor }))
);
```

Wrap usage in `<Suspense fallback={<LoadingSkeleton />}>`. Fallback is a plain `<textarea>` with identical `value`/`onChange`.

5.3 Replace lines 201–213 (the "Compiled Prompt" card)

Structure:

```
<div style={glassCard}>
  <div style={sectionHeader}>Prompt Templates</div>
  <Tabs value={tab} onChange={setTab}>
    <Tab id="system">System Prompt</Tab>
    <Tab id="user">User Prompt</Tab>
  </Tabs>

  <Suspense fallback={<textarea … />}>
    <MonacoEditor
      height="320px"
      language="handlebars"          // closest built-in; gives {{ }} highlighting
      theme="vs-dark"
      value={currentTab === 'system'
        ? (cfg.systemPromptTemplate ?? BUILTIN_SYSTEM)
        : (cfg.userPromptTemplate ?? BUILTIN_USER)}
      onChange={(v) => setCfg(c => ({
        ...c,
        [tab === 'system' ? 'systemPromptTemplate' : 'userPromptTemplate']: v ?? '',
      }))}
      options={{ minimap: { enabled: false }, fontSize: 13, wordWrap: 'on' }}
    />
  </Suspense>

  <ValidationPanel result={validation[tab]} />

  <div style={{ display: 'flex', gap: 8 }}>
    <button onClick={resetToDefault}>Reset to default</button>
    <button onClick={() => setShowPreview(true)}>Preview compiled prompt</button>
    {/* legacy compiled-prompt summary stays as a subtle footer */}
  </div>
</div>

{showPreview && <PreviewModal … />}
```

- `BUILTIN_SYSTEM` / `BUILTIN_USER`: exported from a new helper `src/lib/builtinPromptTemplates.ts` that mirrors the current strings in `backend/src/services/prompt-template.ts` (W1 will promote these into the single source of truth; for now duplicate and TODO).
- "Reset to default": sets the field to `null` (server interprets as "use built-in"), editor re-renders the built-in text read-only.
- Live validation: `useMemo` calling `validateSystemTemplate` / `validateUserTemplate` on every keystroke.

5.4 `ValidationPanel` (inline component)

- Red badge per missing required var (e.g. `Missing {{claim}}`).
- Yellow badge per unknown var.
- Green check "Output schema OK" or red error listing missing keys.
- Aria-live="polite" so screen readers announce changes.

5.5 `PreviewModal`

- Invokes `POST /v1/projects/:projectId/compiled-prompt/preview` (new endpoint — see §6.3) with the *in-flight* (unsaved) template and a synthetic sample claim baked client-side: `ts const SAMPLE = { claim_id: '00000000-0000-0000-0000-000000000001', claim: 'The mitochondria is the powerhouse of the cell.', evidence: [{ title: 'Biology 101', snippet: 'Mitochondria produce ATP.', url: 'https://example.org/bio' }], guideline: 'Prefer peer-reviewed sources.', tier: 'standard', };`
- Renders the returned `{ system, user, promptHash }` in two read-only `<pre>` blocks — this is the final substituted text the LLM would see.

5.6 Save-gate

In the existing save handler, refuse to POST if `validation.system.valid === false || validation.user.valid === false`. Disable the primary Save button; show the first error as a tooltip.

---

6. Backend diffs

6.1 `backend/src/routes/judgeConfig.ts`

6.1.1 Extend `PutBodySchema` (line 50)

```
 const PutBodySchema = z.object({
   provider_id: z.string().uuid(),
   model: z.string().min(1).max(200),
   …
   fallback_chain: z.array(FallbackEntrySchema).max(5).optional(),
+  system_prompt_template: z.string().max(16_000).nullable().optional(),
+  user_prompt_template:   z.string().max(16_000).nullable().optional(),
 });
```

6.1.2 Server-side validation (after body parse, before upsert)

```
import {
  validateSystemTemplate, validateUserTemplate,
} from '../services/promptTemplateValidation.js';

if (body.system_prompt_template != null) {
  const r = validateSystemTemplate(body.system_prompt_template);
  if (!r.valid) {
    return reply.code(422).send({
```

```
      error: 'Invalid system prompt template',
      code: 'ERR-VALIDATION',
      missing_required: r.missingRequired,
      output_schema_ok: r.outputSchemaOk,
    });
  }
}
if (body.user_prompt_template != null) {
  const r = validateUserTemplate(body.user_prompt_template);
  if (!r.valid) {
    return reply.code(422).send({
      error: 'Invalid user prompt template',
      code: 'ERR-VALIDATION',
      missing_required: r.missingRequired,
    });
  }
}
```

Empty string ≠ null — reject empty. `null` means "use built-in".

6.1.3 Extend the upsert SQL

Add the two columns to both the INSERT list and the `ON CONFLICT DO UPDATE SET` clause (near line 541):

```
   INSERT INTO public.project_judge_config
-    (project_id, provider_id, model, ..., fallback_chain, version, updated_at, updated_by)
+    (project_id, provider_id, model, ..., fallback_chain,
+     system_prompt_template, user_prompt_template,
+     version, updated_at, updated_by)
   VALUES (..., ${body.system_prompt_template ?? null},
               ${body.user_prompt_template ?? null}, ...)
   ON CONFLICT (project_id) DO UPDATE SET
     ...,
+    system_prompt_template = EXCLUDED.system_prompt_template,
+    user_prompt_template   = EXCLUDED.user_prompt_template,
     ...
```

Mirror in the history-snapshot INSERT at line 570.

Extend `formatConfig()` (line 191) to expose the two fields:

```
   fallback_chain: row.fallback_chain ?? [],
+  systemPromptTemplate: row.system_prompt_template ?? null,
+  userPromptTemplate:   row.user_prompt_template ?? null,
+  system_prompt_template: row.system_prompt_template ?? null,
+  user_prompt_template:   row.user_prompt_template ?? null,
```

And extend `JudgeConfigRow` and `JudgeConfigHistoryRow` TS interfaces. Update the SELECT lists in both GET handlers (lines 264 and 339).

Audit-log payload (line 629): add `system_prompt_custom: body.system_prompt_template != null` and `user_prompt_custom: body.user_prompt_template != null` booleans. Do **not** log the template body (PII / noise).

6.2 `backend/src/services/prompt-template.ts`

Make `renderJudgePrompt()` optionally take overrides. Backward-compat: existing callers work unchanged.

```
export interface RenderOptions {
  systemTemplateOverride?: string | null;
  userTemplateOverride?:   string | null;
}

export function renderJudgePrompt(
  ctx: ClaimContext,
  opts: RenderOptions = {},
): JudgePrompt {
  const system = opts.systemTemplateOverride
    ? substitute(opts.systemTemplateOverride, buildVarBag(ctx))
    : buildSystemPrompt(buildGuidelineSection(ctx.projectGuideline));
  const user = opts.userTemplateOverride
    ? substitute(opts.userTemplateOverride, buildVarBag(ctx))
    : buildUserPrompt(ctx);
  // … existing hash computation
}

function buildVarBag(ctx: ClaimContext) {
  return {
    claim:     truncate(ctx.claimText, MAX_CLAIM_CHARS),
    claim_id:  ctx.claimId,
    tier:      ctx.tier,
    guideline: truncate((ctx.projectGuideline ?? '').trim(), MAX_GUIDELINE_CHARS),
    evidence:  renderEvidenceBlocks(ctx.evidence),  // existing logic extracted
  };
}

function substitute(tmpl: string, bag: Record<string, string>): string {
  return tmpl.replace(/\{\{\s*(\w+)\s*\}\}/g, (_, k) => bag[k] ?? '');
}
```

Truncation still applied to substituted vars. Injection guard (the delimiter-stripping behavior) stays automatic because the substituted `evidence` block still wraps each item in `<<<EVIDENCE_N>>>`.

The caller (LLM-run dispatcher, outside scope of this ticket) must look up `project_judge_config.system_prompt_template` / `user_prompt_template` and pass them in. Coordinate with W1 owner — put a TODO in `prompt-template.ts` referencing this ticket.

6.3 New endpoint: `POST /v1/projects/:projectId/compiled-prompt/preview`

Belongs in `backend/src/routes/compiledPrompt.ts`.

Body:

```
{ system_prompt_template: string | null,
  user_prompt_template:   string | null,
  sample: { claim_id, claim, evidence, guideline, tier } }
```

Handler: 1. `requireRole('ADMIN','PM')`. 2. Validate templates via the shared module (same 422 shape as §6.1.2). 3. Call `renderJudgePrompt(sample, { systemTemplateOverride, userTemplateOverride })`. 4. Return `{ system, user, promptHash }` — **never persisted**, no audit row (preview is idempotent and high-volume). 5. Rate-limit: 30 / minute / user (in-memory), shape reused from the existing `/test` limiter.

---

7. Accessibility & UX notes

- Monaco is intrinsically keyboard-navigable; ensure our Tab component between System/User uses `role="tablist"` and arrow-key navigation.
- The `<textarea>` fallback must have a visible label, not placeholder-only.
- Contrast: Monaco's `vs-dark` against our glassmorphism background is OK; use a border to separate them visually.
- Preview modal traps focus, `Esc` closes.

---

8. Migration & rollout

1. Merge migration (§3) first; `system_prompt_template` defaults to NULL so older backend code keeps working (reads the column but ignores it).
2. Merge backend (§6). Existing clients that don't send the two fields see no change (omitted optional → `null` → built-in used).
3. Merge frontend (§5). Feature is additive — no flag required.
4. E2E: add a Playwright spec `tests/e2e/judge-config-templates.spec.ts`:
5. PM opens JudgeConfig → switches to Custom templates tab → types an invalid template (missing `{{claim}}`) → Save disabled.
6. PM types a valid template → clicks "Preview compiled prompt" → sees substituted output containing `<<<CLAIM>>>`.
7. PM saves → reloads → template persists.
8. PM clicks "Reset to default" → field becomes null → compiled prompt again shows the built-in.

---

9. Non-goals

- **Diff viewer between versions.** History lists versions, but side-by-side template diff is out of scope (log a follow-up ticket).
- **Syntax-aware linting (e.g. JSON-schema hints inside the template).** Monaco language is `handlebars`, no custom tokens provider.

- **Per-tier overrides.** One template per project, not per tier.
- **Template marketplace / sharing across projects.** Copy-paste for now.

---

10. Risk register addenda

Risk	Mitigation
Monaco CDN unavailable offline	textarea fallback (§2.1) keeps save path functional.
PM writes a template that passes var validation but omits the JSON output instruction	Output-schema check (§4.1, item 6) catches missing rubric keys.
Server renderer and client validator drift	Single shared module (§4.1); duplication between `src/lib/` and `backend/src/services/` is temporary — file one follow-up to extract a workspace package.
Token blow-up from a verbose custom template	`max(16_000)` char cap at zod layer; truncation on substituted vars (§6.2).
Old `llm_runs` rows reference a deleted historical template	`_history` table is append-only; no DELETE path exposed.

---

11. Acceptance criteria

- [ ] `@monaco-editor/react` appears in `package.json` dependencies.
- [ ] Migration applies cleanly to a fresh DB and to a DB with existing rows (nullable default).
- [ ] `JudgeConfig.tsx` renders two tabs; Monaco loads lazily (verified via a Vite build reporting a separate monaco async chunk).
- [ ] Validation panel shows live red/yellow/green states.
- [ ] "Reset to default" sets the field to NULL on save.
- [ ] "Preview compiled prompt" returns substituted text matching `<<<CLAIM>>>` / `<<<EVIDENCE_1>>>` structure.
- [ ] PUT with invalid template returns 422 with a descriptive body.
- [ ] `project_judge_config_history` captures the two new columns on every save.
- [ ] Vitest suite for `promptTemplateValidation` passes.
- [ ] Playwright template-editing spec green.
- [ ] `npm run lint` clean; `npm run build` clean; `backend/npm run test` green.